

Satellite Constellation Simulator

A Python-based simulation tool for generating, propagating, and visualizing satellite constellations (specifically Walker Delta patterns). This tool uses the **Skyfield** library for high-fidelity SGP4 propagation and **Matplotlib** for rendering 3D orbits, ground tracks, and sky views.

Quick Start

1. Install Dependencies

```
pip install -r requirements.txt
```

2. Run a Simulation

The script automatically detects if you are on a server (headless) or a desktop.

Generate a Sky View from the Azores:

```
python3 constellation_sim.py sky --location azores --trails --save
```

Output: skyview_output.gif

Generate a 3D Orbital View:

```
python3 constellation_sim.py orbit --save
```

Output: orbit_3d_output.gif

🔧 Usage & Arguments

The script `constellation_sim.py` accepts several command-line arguments to customize the physics and the view.

Command Structure

```
python3 constellation_sim.py [VIEW_MODE] [OPTIONS]
```

View Modes

Mode	Description
sky	Observer View (Topocentric). Simulates what a person on the ground sees looking up. Useful for coverage analysis.

orbit	Space View (ECI). A 3D plot of the Earth and the satellites orbiting around it. Useful for constellation geometry.
track	Map View (Mercator). Plots the "Ground Track" (path over the surface). Useful for seeing Earth rotation effects.

Options

Flag	Default	Description
--sats	12	Total number of satellites in the constellation.
--planes	3	Number of orbital planes to split the satellites into.
--inc	53.0	Orbital inclination in degrees (0 = Equator, 90 = Polar).
--alt	560	Orbit altitude in Kilometers.
--location	lisbon	Observer location for Sky View (e.g., azores , tokyo , rio).
--trails	False	(Sky View only) Draw gray lines tracing the satellite's path.
--save	Auto	Force saving to GIF. (Auto-enabled on headless servers).

Examples

1. High-Density Polar Constellation (OneWeb style) Simulate 100 satellites in 10 planes at 90° inclination.

```
python3 constellation_sim.py orbit --sats 100 --planes 10 --inc 90 --save
```

2. Coverage Check for Tokyo Check visibility trails from Tokyo.

```
python3 constellation_sim.py sky --location tokyo --trails --save
```

 Orbital Mechanics Syllabus

This simulation is based on real astrodynamics. Here is a primer on the concepts used in the code.

1. The Walker Delta Pattern

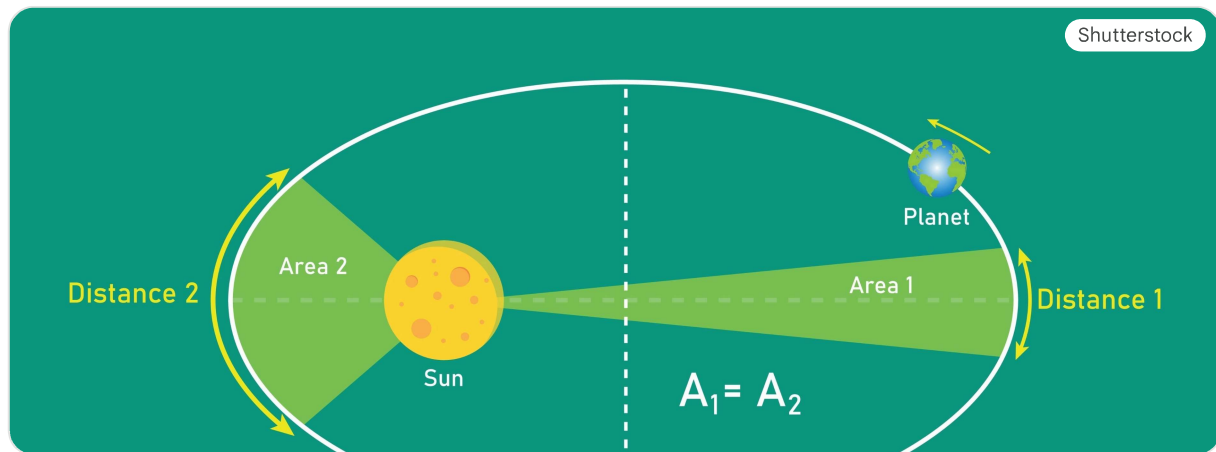
Most modern internet constellations (Starlink, OneWeb, Galileo) use a **Walker Delta** pattern. It is denoted as: $i : t / p / f$

- i (**Inclination**): The tilt of the orbit relative to the Equator. 53° is common because it covers most populated areas while avoiding the radiation at the poles.
- t (**Total Satellites**): The total count.

- **p (Planes):** Satellites are not all in one ring; they are split into several rings (planes) rotated around the Earth.
- **f (Phasing):** This is the "offset" parameter. It ensures that satellites in adjacent planes don't collide at the poles and maximizes coverage mesh.

2. Keplerian Elements vs. TLE

To simulate an orbit, we need 6 parameters (Keplerian Elements):



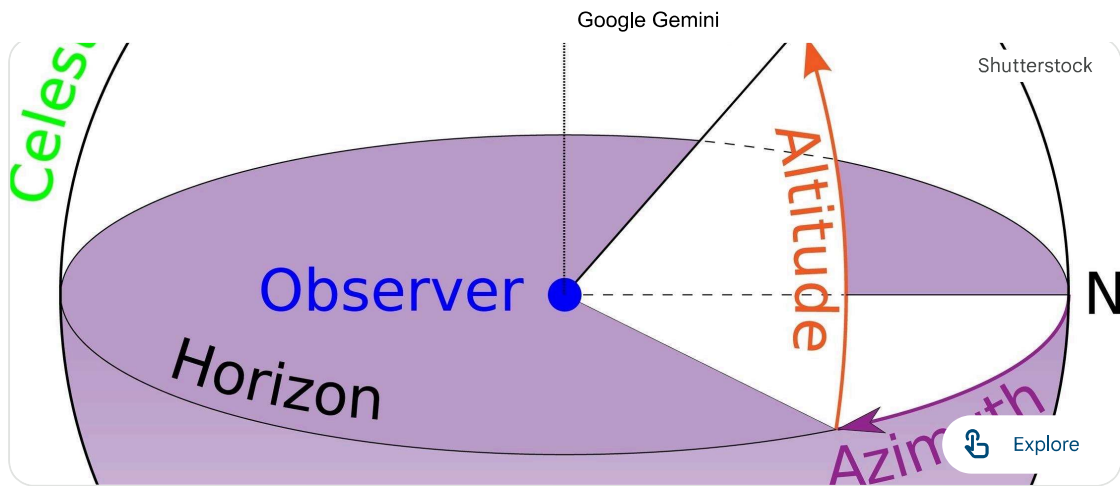
1. **Semi-Major Axis (a):** Size of the orbit (Altitude + Earth Radius).
2. **Eccentricity (e):** Shape (0 = Circular, which we use).
3. **Inclination (i):** Tilt.
4. **RAAN (Ω):** *Right Ascension of Ascending Node*. The angle of the plane around the Earth's rotation axis.
5. **Argument of Perigee (ω):** Orientation of the ellipse (doesn't matter much for circular orbits).
6. **Mean Anomaly (M):** Where the satellite is *right now* on the circle.

What the script does: We calculate these 6 values mathematically for every satellite, then convert them into a TLE (**Two-Line Element**) string. TLE is the industry standard format used by NORAD to track space objects.

3. Coordinate Frames

The simulation handles three different "points of view":

- **ECI (Earth-Centered Inertial):** Used in `orbit` view. The grid is fixed to the stars. The Earth rotates inside this grid. This is why the satellites look like they follow perfect loops.
- **ECEF (Earth-Centered Earth-Fixed):** Used in `track` view. The grid spins *with* the Earth. Because the Earth spins underneath the satellite, the satellite appears to drift Westward on every orbit.
- **Topocentric (Alt/Az):** Used in `sky` view.
 - **Azimuth:** Compass direction (0° N, 90° E).
 - **Elevation:** Height in sky (0° Horizon, 90° Zenith).



4. SGP4 Propagation

We do not use simple Kepler equations ($Pos = Vel \times Time$) because they are inaccurate for Earth orbits. We use SGP4 (Simplified General Perturbations 4) via the `skyfield` library. This model accounts for:

- Earth's oblateness (it bulges at the equator, known as J_2 perturbation).
- Atmospheric drag (simplified)